What are Structured Types (Concept)

In advanced SQL (especially Object-Relational DBMS like PostgreSQL, Oracle):
A structured type = a user-defined type made up of multiple fields (like a class in programming).

```
CREATE TYPE Address AS (
  street VARCHAR(50),
  city VARCHAR(30),
  pincode INT
);
```

Then you can use it:

```
CREATE TABLE Student (
  id INT,
  name VARCHAR(50),
  addr Address
);
```

Here addr is a structured type (like an object)

2. Important Point (MySQL Limitation)

MySQL DOES NOT support structured types directly like the above.
So you cannot create custom composite types using CREATE TYPE in MySQL.

Then how do we achieve this in MySQL?

We simulate structured types using 3 approaches:

Approach 1: Normalized Tables
This is the most correct relational way.
Example CREATE TABLE Address (
  address_id INT PRIMARY KEY,
  street VARCHAR(50),   city VARCHAR(30),
  pincode INT);

CREATE TABLE Student (id INT PRIMARY KEY,
  name VARCHAR(50),   address_id INT,
  FOREIGN KEY (address_id) REFERENCES
Address(address_id)
);
structured types using table relationships
This is what we teach in DBMS as Normalization

Approach 2: JSON Data Type (Modern MySQL)
MySQL supports JSON → closest to structured type behavior.
Example:
CREATE TABLE Student (
  id INT,
  name VARCHAR(50),
  address JSON
);
Insert:
INSERT INTO Student VALUES (
  1,
  'Rahul',
  '{"street":"MG Road", "city":"Kochi",

```
"pincode":682001}'
);
```

Query:

```
SELECT address->'$.city' FROM Student;
```

JSON behaves like a nested structured object

Approach 3: Embedded Columns (Simple but less flexible)

```
CREATE TABLE Student (
  id INT,
  name VARCHAR(50),
  street VARCHAR(50),
  city VARCHAR(30),
  pincode INT
);
```

Easy

Not reusable (no abstraction like structured types)

Feature	Structured Types (Theory)	MySQL Reality
Custom type creation	✔ Yes	✖ No
Nested structure	✔ Yes	✔ Using JSON

Feature	Structured Types (Theory)	MySQL Reality
Reusability	✔ High	✔ via tables
Object-like behavior	✔ Yes	⚠ Partial